Policy Engine Subsystem

This architecture is for phase 1, and phase 2 compatible. Phase 2 architecture will be extended based on this, with clustering support added.

- 1. Overview
  - 1.1. Intro
  - 1.2. Policy modeling
  - 1.3. Logical View
  - 1.4. Policy Role and Responsibilities & Policy Implementation
  - 1.5. Data Ownership
- 2. Role and Responsibilities
  - 2.1. Out-of-scope
- 3. Resource & Data in the subsystem
- 4. Interfaces
  - 4.1. Incoming
    - 4.1.1. REST
    - 4.1.2. Event Subscriptions
  - 4.2. Outgoing
    - 4.2.1. REST
    - 4.2.2. Notifications
- 5. Scale Target
- 6. Domains and interactions
- 7. Domain overview
  - 7.1. Policy
    - 7.1.1. Phase 1 work
    - 7.1.2. Resilience targets
    - 7.1.3. Performance targets
  - 7.2. Central interpreter
    - 7.2.1. Phase 1 work
    - 7.2.2. Resilience targets
  - 7.3. Execution Planner
    - 7.3.1. Phase 1 work
    - 7.3.2. Resilience targets
  - 7.4. Scheduler
    - 7.4.1. Phase 1 work
    - 7.4.2. Resilience targets
  - 7.5. Creator
    - 7.5.1. Phase 1 work
    - 7.5.2. Resilience targets
  - 7.6. Dispatcher
    - 7.6.1. Phase 1 work
    - 7.6.2. Resilience targets
    - 7.6.3. Performance Target
  - 7.7. Executor
    - 7.7.1. Phase 1 work
    - 7.7.2. Resilience targets
    - 7.7.3. Performance Target
  - 7.8. Activity
    - 7.8.1. Phase 1 work
    - 7.8.2. Performance Target

1. Overview

1.1. Intro

Policy Engine is the control plane of data protection control plane. It automate the daily data protection operations and provide observabilities to the customer.

Policy Engine covers

- Policy experience and usability
- Job submission (adhoc or scheduled w/o Policy)
- Policy implementation (or fulfillment) and scheduling
- Job monitoring
- Job intervention (e.g., cancel)

To improve the pNPS/cNPS and gain market share, we need a powerful Policy Engine with enough agility, to fulfill whatever the fields' asks, fast with quality

- Fast TTM to onboard new workload or extend capabilities on the existing workloads
- Support different deployments (PPDM standalone, IDPA++, PPDM-DDVE, Santorini and APEX) with the same code base and user experience
- Quickly adjust the user experience and/or product behavior per customer feedback (even back and force)
- Provide enough flexibility while keep simplicity
- Support larger scale (vertically and horizontally)

While in the other hand, reduce the touchpoints and cost

- Less dependencies between teams and stakeholders thus less planning and management overhead
- Less development efforts
- Less resource footprint (CPU and Mem)
- Less bug/regression
- Less DUDL or disruption

To achieve that, we need to provide the core platform with the architecture of

- Clear role and responsibility breakdown and boundaries
- Clear protocols for components to collaborate. Elastic enough to handle all error cases. Predictable and reasonable behavior for any kind situation
- Pluggable architecture so it's easier to extend
- Self-contained logic so it's easier to change/adjust
- Predictable CPU/Memory usage and performance
- Observable and traceable

Policy Engine is a proactive system. Besides the interfaces to the customer, it proactively do the heavy lift under the scene. E.g, job scheduling and tracking. It's much complex and heavy than a passive system. We need to divide and conquer.

The system can be divided from two dimensions,

- Rich and leading industry functionalities (more PM driven)
    - Abstract the functionalities of current, up-coming and future and break them into different directions where they can grow independently
    - Grow them with flexibility to use or change while keep simplicity to the customer
- Modern micro-services (more engineer driven)
    - Abstract the domain logics and decompose them into loosely coupled micro-services
    - Implement the micro-services in a way that follows the guidelines and best practices in the industry
    - Smoothly move from current architecture to this

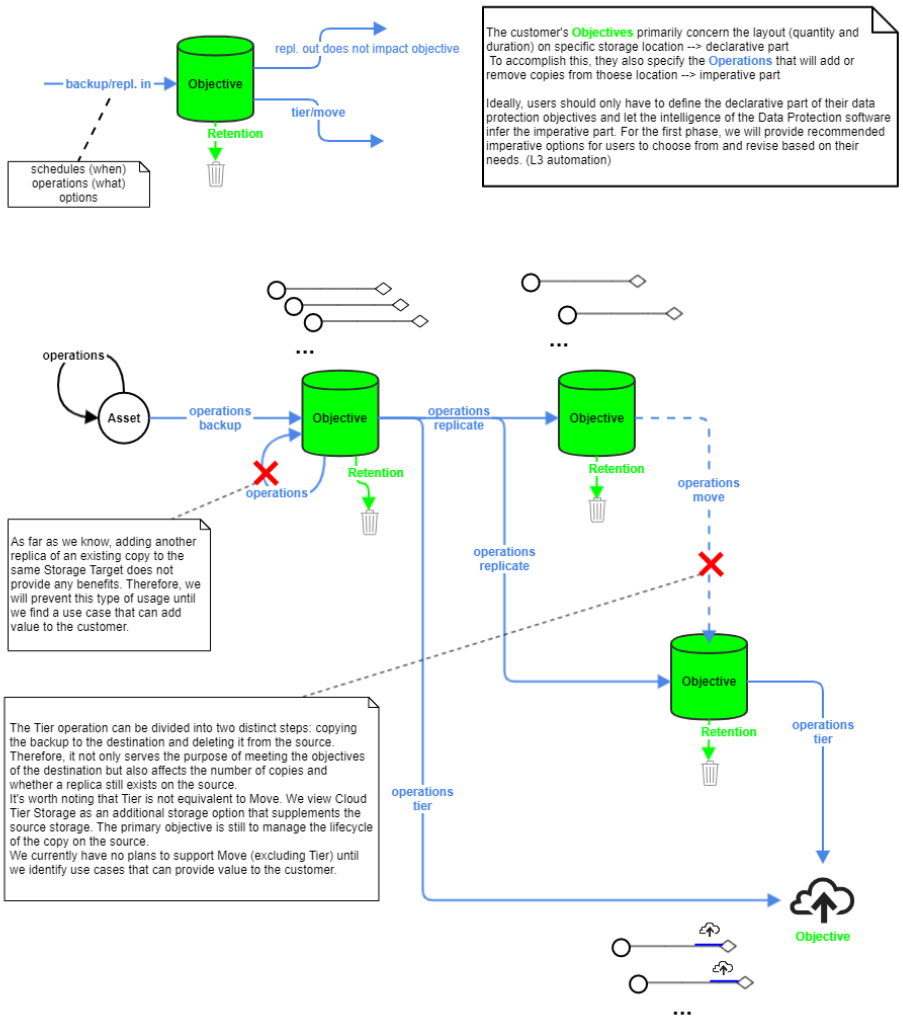
1.2. Policy modeling

Policy is the vehicle to define their goals regarding the life cycle of the copies of the assets applied or to be applied.

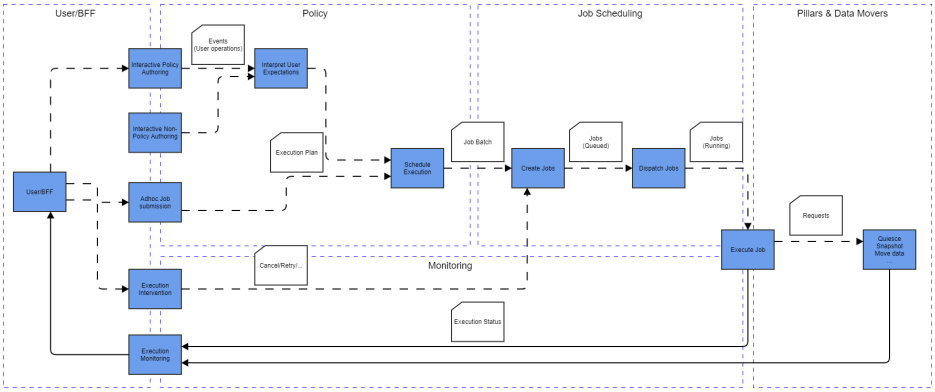
Policy defines

1. where the copies to be placed (Storage Target)
2. how to put the copies there (operations)
    a. when (schedules)
    b. operations (what) with
    c. options (how)
3. and how long the copies will be retained there (Retention)

The below diagram illustrate the relations of these concepts.



1.3. Logical View



1.4. Policy Role and Responsibilities & Policy Implementation

Policy/Adhoc-request ..... Running Job Completed Job

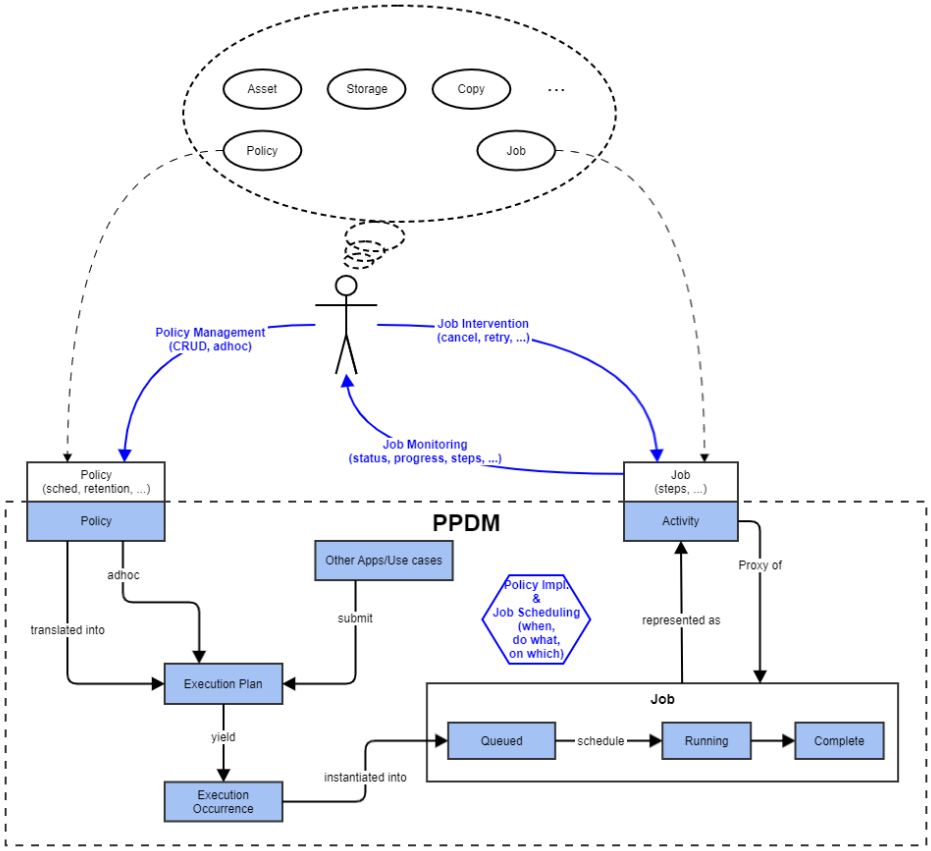
| Execution Plan Execution Occurrences (job batch) QueuedJob |

| Two-phased Policy Impl. | Mon./Intervention |

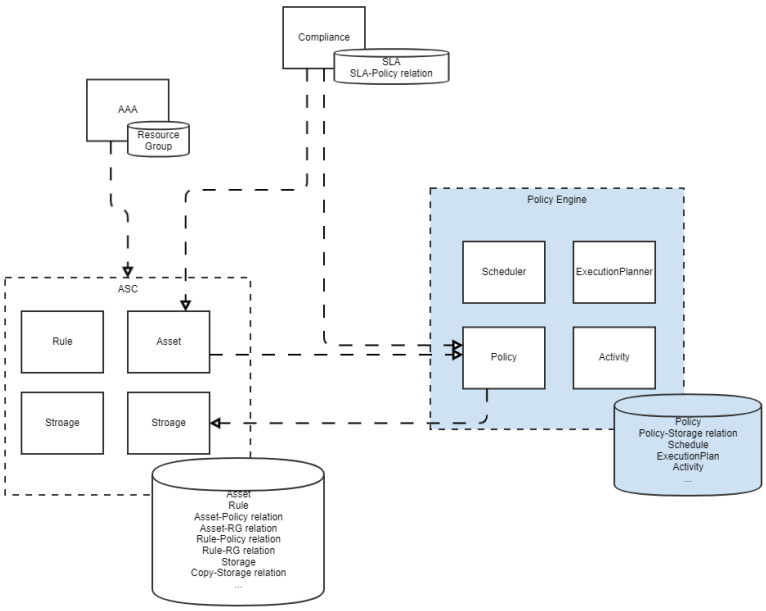
Two-phased Policy Impl.

Policy directly to Running Job is counterintuitive. Based on the current PPDM Activity structure and the experience of MS outlook, a two-phased approach was introduced.

- Phase 1 - Policy to Job batch: Resolve schedule (occurrence) level conflicts **within a Policy**. A MS calendar like history and forecasting can be provided.
- Phase 2 - Job dispatching: Resolve job dependencies and competitions between jobs **across Policies**.



1.5. Data Ownership



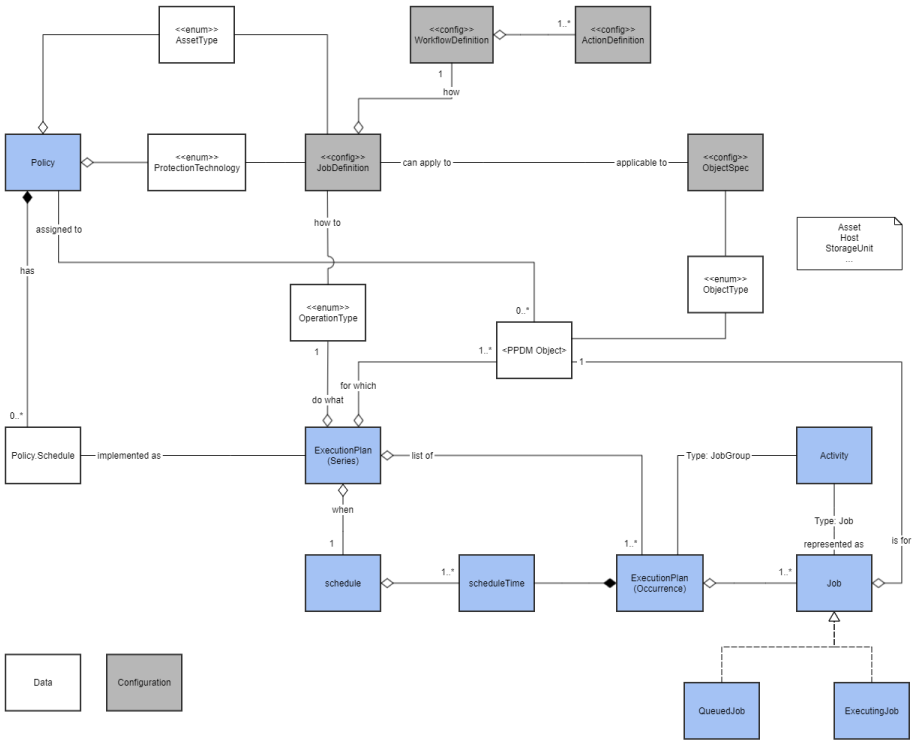
2. Role and Responsibilities

- Policy authoring and management
- Calendar view of Execution Plan for a given Policy and/or Asset
- Submit and manage Execution Plan for Jobs to run immediately, or once at a future time, or periodically. Cancel or move certain occurrences of an Execution Plan
- Translate user operations into PPDM workflows (e.g., password change of a storage target)
- Scheduling and execution of Policy, Execution Plan and Job
- Job dispatching and throttling
- Job monitoring
- Job execution intervention (Cancel, Pause/Resume, Realtime Monitoring, etc.)

2.1. Out-of-scope

- Persist and maintain Asset - Policy relation (neither static, nor dynamic ASC)
- Alerts and Notifications to the customer (System/Platform services. TBD)

3. Resource & Data in the subsystem



Resource/Data	Data or Config	Owner Domain	External	Interfaces	Yield Notifications	Storage
Policy	Data	policy	Y	REST	All Events	ES
Activity	Data	activity	Y	REST	All Events	ES + redis (for transient data. e.g., progress)
Schedule	Data	scheduler	N (to be validate)	REST	All Events	Quartz (postgres as backend)
ExecutionPlan	Data	execution-planner	N (in phase 1)	REST	All Events	ES
QueuedItem (queued Job)	Data	job-dispatcher	N	Internal (RESTful, thru REST compatible client)	No	ES
ExecutionItem (running Job)	Data	job-executor	N	Internal (RESTful, thru REST compatible client)	State changed	Camunda (postgres as backend)
Asset Specification	Configuration					
Object Specification	Configuration					
Job Definition	Configuration					
Workflow Definition	Configuration	job-executor	N	REST (GET only)		
Activity Definition	Configuration	job-executor	N	REST (GET only)		

4. Interfaces

4.1. Incoming

4.1.1. REST

Note: all new APIs (aka, V3) will be moved to the centralized api-repo when they became stable.

Resources	Operations	API Specifications	Comments
Policy	CRUD	Current: https://eos2git.cec.lab.emc.com/data-manager/ppdm-api-publication/blob/master/src/apispecs/protection-policy.yaml .Next: WIP https://eos2git.cec.lab.emc.com/zhangr18/ppdm-policy-engine-next-gen-apis/tree/master/models/protection-policy	

ExecutionPlan	batch-execution CRUD -- out of phase 1 -- executionPlan/{id}/instances calendarView instances/{id}/cancel instances/{id}/move	Current: N/A (New) .Next: WIP https://eos2git.cec.lab.emc.com/data-manager/ecdm/blob/master/cs/protection-policy/execution-planner/execution-planner-codegen/src/main/resources/api-specs/execution-planner.yaml	Refer to O365 calendar APIs Add "None-schedule" support for "Run After" and "Run with" (e.g., replication cases)
Activity	CRUD step-update, progress-update cancel, retry, realtime-monitoring	Current: https://eos2git.cec.lab.emc.com/data-manager/ppdm-api-publication/blob/master/src/apispecs/monitoring.yaml .Next: WIP	
Schedule (Policy Engine internal)	CRUD query schedule occurrences	Current: https://gitdpsg.cec.lab.emc.com/projects/ECDM/repos/ppdm-api-publication/browse/src/apispecs/scheduler.yaml .Next: WIP https://eos2git.cec.lab.emc.com/data-manager/ppdm-api-publication/blob/master/src/apispecs/scheduler.v3.yaml	Refer to O365 and Azure scheduler APIs Add "minutely/hourly" support

4.1.2. Event Subscriptions

Asset created/changed/deleted

Policy created/changed/deleted

AssetAssignment changed

Credential changed

RestoredCopies/... created/deleted

Asset source deleted

System-level configuration update (for example encryption over the wire)

4.2. Outgoing

4.2.1. REST

Asset, Storage

Resource Management APIs

Pillar configuration/execution APIs

4.2.2. Notifications

Policy created/changed/deleted

ExecutionPlan created/changed/deleted

Activity created/changed/deleted

PendingCancellation

5. Scale Target

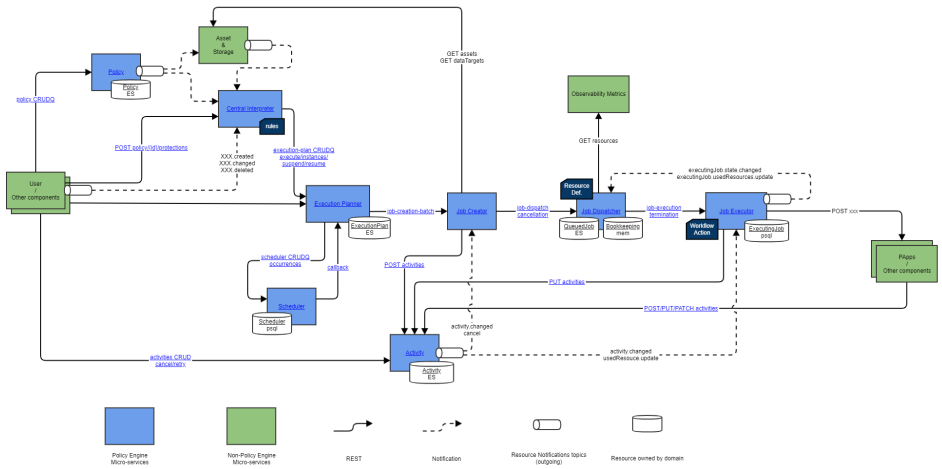
	Ithaca	Rhodes (PPDM.Next phase 1)	PPDM.Next phase 2a (single node)	PPDM.Next phase 2b (clustered)
Assets	10K	15K	15K	150K
Jobs (copies)	100K	150K	3M	30M

Parallel jobs (backup + repl.) NOTE: deletion and other jobs was not counted	1K	1K	TBD	TBD
Min RPO	15min	15min	15min	15min

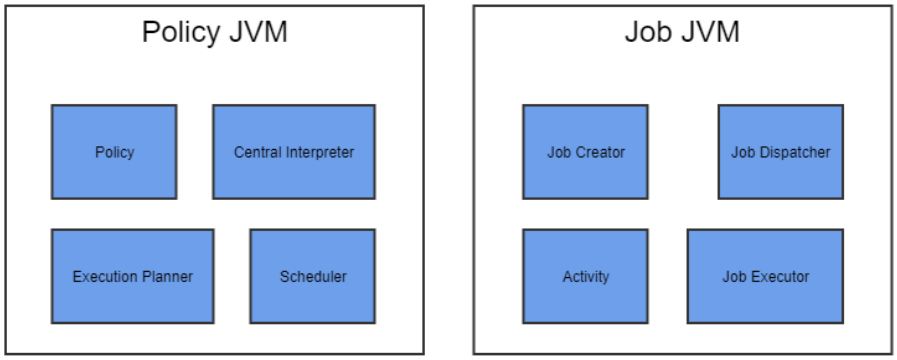
References: DP Series Scale

6. Domains and interactions

Domains - phase 1

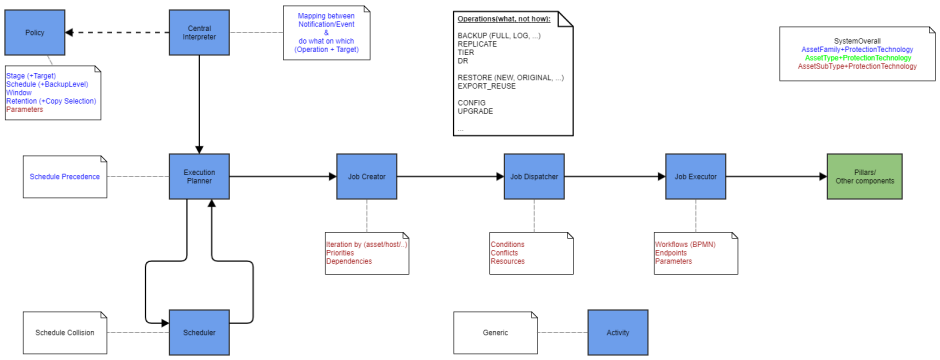


Physical Deployment - phase 1

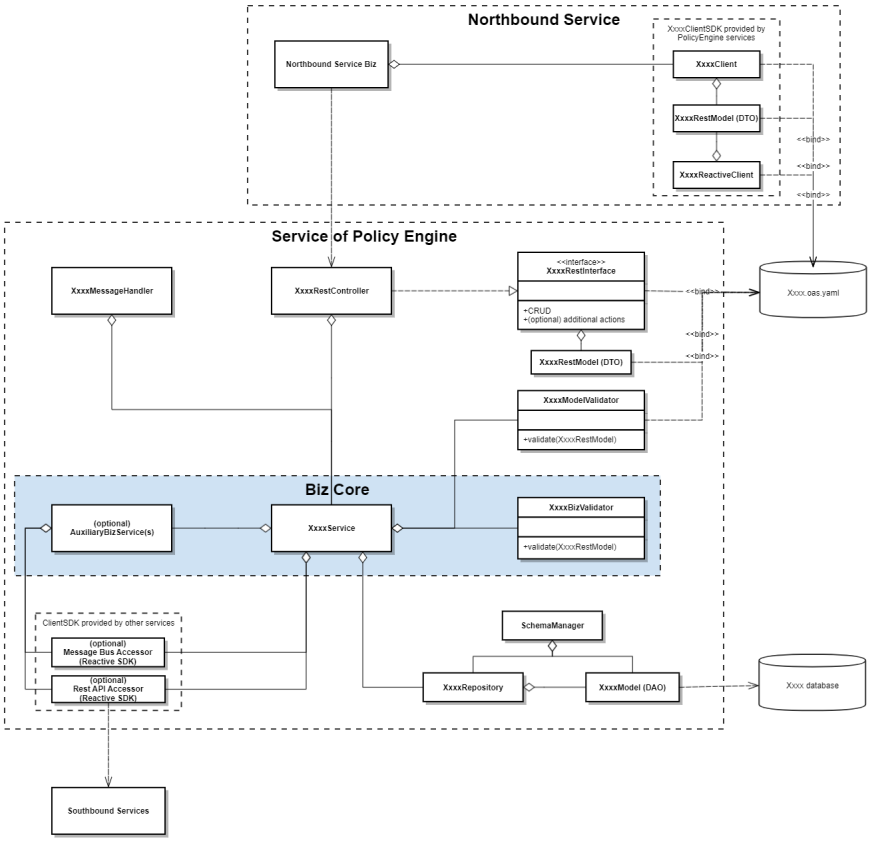


previous deployment is here

Awareness of Asset commonalities and differences in different micro-services



Common structure of Policy Engine services



7. Domain overview

Scopes in phase 1:

1. Build all Policy Engine business logic into micro-services
    a. Business logic decomposition (so we don't need to re-arch in Phase 2 for phase 2 requirements)
    b. All APIs (for user and/or other non-PolicyEngine services) should be RESTful thru HTTP. Deprecate CRS accordingly (no more cross service lib/DB access)
    c. Use spring-reactive as framework
    d. Self-managed schema upgrade and data migration
2. Match the existing performance and scale
3. Resilience hardening to cover error cases in phase 1 deployment model (timeout, downstream service na, etc.)
4. Micro-service side changes for observability (Defer infrastructure/platform part to phase 2)

Strive for:

1. Exceed the existing performance and scale
2. Less CPU & Memory footprint

Out of scope:

1. Atlantic/K8s integration
2. Scale-out
3. NDU and Rolling-upgrade
4. Scale-in
5. Auto-scale

7.1. Policy

Policy Service is the owner of "policy" resource. It provides REST APIs for the user to do management of their Policy and emit notifications to the subscribers.

For any operations on "policy", Policy service does certain validations then record the changes. All the fulfillment will be deferred to other components and processed asynchronously.

Fully API driven. No more library calls (CRS/DAL).

AssetAssignment APIs move to ASC

Reverse the dependencies between Policy and SLA (Policy will not be aware of SLA existence) in V3 APIs

7.1.1. Phase 1 work

1. Data ownership of "Policy" and REST APIs
2. Migrate to spring-reactive framework
3. SDK for consumer
4. Observability
5. Migrate all consumers from CRS to SDK. Disable/deprecate Policy CBS/CRS
6. Resilience hardening

7.1.2. Resilience targets

Max down time < 30s (time from restart to READY)

7.1.3. Performance targets

	Throughput (rps)	Latency (ms of 99 percentile)
Get by ID	1k	10
Others	200	200

7.2. Central interpreter

Central interpreter subscribes notifications of Policy and other sources, and process them asynchronously. Translate them into "events" then into "ExecutionPlan" according to the pre-defined "rules" (spec-driven). E.g.: Policy.changed ScheduleAdded ExecutionPlan for the new Schedule.

ExecutionPlan will later be translated into Jobs.

Central interpreter does NOT have any data persistence.

Based on former DPS

Build the framework to handle more generic notifications rather then specific (Asset.Changed vs AssetConfiguration) to fully leverage spec driven framework we have done.

Use ExecutionPlan APIs as the downstream interface instead of workflowResource and workflowExec

Migrate all former DPS handled workflows to the next framework

7.2.1. Phase 1 work

1. Migrate to spring-reactive framework
2. Migrate from (workflow + schedule) to ExecutionPlan

7.2.2. Resilience targets

Max down time < 2m (time from restart to READY)

7.3. Execution Planner

Execution Planner Service is the owner of "ExecutionPlan" resource. It provides REST APIs to manage "ExecutionPlan" and emit notifications to the subscribers.

Execution Planner also provides a dedicate API for adhoc execution (Special "Run Now" ExecutionPlan which does not return an instance of ExecutionPlan but just a Job Group ID)

Execution Planner resolves conflicts between multiple "ExecutionPlans" (or "Execution Occurrence") according to pre-defined "precedence rules". E.g., "Monthly" > "Weekly" > "Daily", but LOG can run together with FULL (The schedule collision handling was a collaboration of Scheduler and Execution Planner)

Execution Planner submits the batch (batches in some case. e.g., FULL + LOG) of Jobs of a certain occurrence of "ExecutionPlan" to Creator.

See the difference to deal with collision, precedence and dependencies in phase 1 vs R8

----- out of phase 1 scope ----

Calendar view of "ExecutionPlan" for given "ExcutionPlan" or all "ExecutionPlan" of a given Policy/Asset

List occurrences of "ExecutionPlan". Cancel/Move occurrence of "ExecutionPlan"

New in phase1

Model and APIs will align to O365 outlook calendar APIs

Besides O365 outlook calendar capabilities, added non-schedule trigger types ("Run After" and future "Run with") as well as adhoc execution ("Run Now")

Collision handling

Migrate all other workflows which is not handled in Central Interpreter. No more library calls on workflowResource and workflowExec (CRS/DAL).

7.3.1. Phase 1 work

1. Data ownership of "ExecutionPlan" and REST APIs
2. REST API for adhoc execution
3. Create schedules thru Scheduler API
4. REST API for Scheduler callback and submit batches to Creator thru Creator SDK
5. Resolve collisions of occurrences from multiple "ExecutionPlan" based on precedence rules (driven by asset spec)
6. SDK for consumer
7. Observability
8. Migrate all external workflow and/or schedule consumer to ExecutionPlan. Git rid of workflowResource
9. Resilience hardening

7.3.2. Resilience targets

Max down time < 30s (time from restart to READY)

7.4. Scheduler

Scheduler Service is the owner of "schedule" resource. It provides REST APIs for the other components to manage schedule and "window" on schedules.

Scheduler Service do resolution on "schedule" and/or "window" collisions. All the schedules within a "Schedule Group" and emit only one notifications for this group. E.g., "Monthly FULL" + "Weekly FULL" + "Daily FULL" + "Hourly INC" + "Minutely LOG" collide at the same time emit only one notification

Scheduler Service will call the callback API per the definition of a "schedule" when one or a group of schedules arrived or arrived together.

---- out of phase 1 scope ----

List occurrences of "Schedule". Cancel/Move occurrence of "Schedule"

Both CRUD and callback will be RESTful thru HTTP

Model and APIs will align to O365 outlook calendar APIs

Collision detection

7.4.1. Phase 1 work

1. Data ownership of "Schedule" and REST APIs
2. Migrate to spring-reactive framework
3. Migrate Schedule collision handling from per Policy to per "Schedule Group"
4. Callback thru REST
5. SDK for consumer
6. Observability

7. Migrate all consumers from CRS to SDK. Disable/deprecate Schedule CBS/CRS
8. Resilience hardening

7.4.2. Resilience targets

Max down time < 2m (time from restart to READY)

Fire all "misfired" triggers after restart (ZERO misfire)

7.5. Creator

Job Creator Service accepts batch execution request, iterates the workflow on each object (asset, policy, etc.) with given parameters and create jobs accordingly. It also populates dependencies to Jobs when necessary and set priorities on Job per the input.

Job Creator will skip jobs in some cases (e.g., if it was asked to install VIB on a list of hosts but found some of them have already been installed, the jobs will be skipped for these hosts).

Job Creator can automatically recreate a job and requeue it based on pre-defined conditions if the previous job execution failed at any point. (i.e., Job level conditional auto-retry)

Job Creator will also make sure the job which created by it ("owner") can be canceled properly.

REST API to accept "JobBatch"

Get rid of workflowResource

More generic rather than "Asset" centric

7.5.1. Phase 1 work

1. REST API to accept "JobBatch"
2. Migrate to spring-reactive framework
3. Iterate per target "Object" and create Jobs
4. Build Job dependencies if received two "JobBatch" together
5. Create Activates thru Activity SDK
6. Send Job to dispatcher thru Dispatcher SDK
7. SDK for consumer
8. Observability
9. Resilience hardening

7.5.2. Resilience targets

Max down time < 30s (time from restart to READY)

7.6. Dispatcher

Pulling resource status from specified endpoints.

Based on the current resource status and running jobs, decide "which" and "how many" jobs to be executed

Dispatcher maintains a job queue. All the jobs has been created will be staged in the queue first.

The Jobs can be categorized into multiple "Virtual Queues" (VQ) based on the job characteristics (e.g., resource types required)

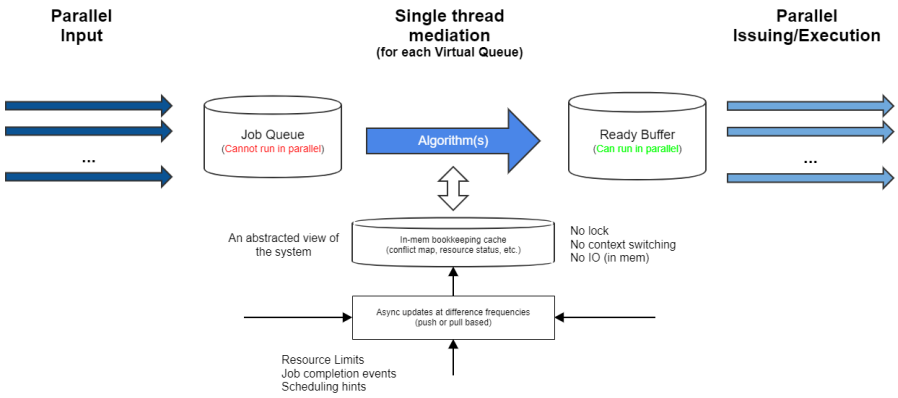
For each "Virtual Queue", a single thread mediator will make sure the job is eligible to run based on

1. time (e.g., not before xxx)
2. dependencies (e.g., replicate after backup)
3. priorities (e.g., restore > adhoc backup > scheduled backup)
4. conditions (e.g., policy/asset have not been configured yet)
5. conflicts (e.g., cannot run another FULL while one already running)
6. resource status (e.g., vproxy capacity)

The jobs were created with dependencies and conflicts (already populated on Jobs). To resolve them, we need to do mediation. Mediation needs strong consistency thus transactional. To reach high throughput, we'll using single thread to do mediation for all the jobs of a "Virtual Queue".

"Virtual Queue" (VQ in short) is a concept which is introduced to improve parallelism in this condition. We can categorize the jobs which might have conflicts into multiple VQs so the conflicts are within the VQ.

- Dependencies can cross VQs. A job in one VQ can depend on other jobs in other VQs.
- Conflicts technically should not cross VQs. One special conflicts is competition of the shared resources. We can divide resource with quota so we can leverage multiple VQs to achieve higher parallelism thus throughput. The quota can be adjusted by the ExeuctionPlanner per the forecast of Jobs.



See mediation pipeline in phase 1 vs R8

SDK for enqueue

Become more generic and spec driven

Move conflict check from executor to dispatcher and check before allocate resource

Move from high IO to in-mem bookkeeping

Use redis with persistence for the queue

7.6.1. Phase 1 work

1. API for enqueue
2. Migrate to spring-reactive framework
3. Optimized data persistence for Job Queue and Ready Buffer
4. Build time and space efficient data structure for bookkeeping
5. Able to rebuild bookkeeping data from Activity and resource provider info
6. Handle executor notifications instead of Activity notification to update bookkeeping
7. Move conflict check from executor to dispatcher and move it before resource allocation
8. Send Job to Executor thru Executor SDK
9. SDK for consumer
10. Observability
11. Resilience hardening

7.6.2. Resilience targets

Max down time < 1m (time from restart to READY)

7.6.3. Performance Target

	Throughput (jobs per sec)
Job Dispatching per VQ	100

7.7. Executor

Job Dispatcher tells Job Executor "what" to do and Job Executor knows "how" to do it.

Job Executor Service issues requests and control the flow based on the workflow definitions and parameters. It send notification to Job Dispatcher for resource usages.

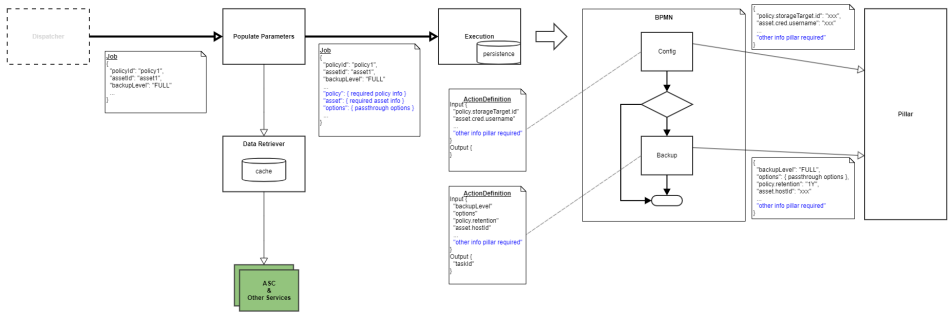
Job Executor can retry a certain failed step by resending the request with

1. Exactly the same request or
2. An redacted request

Based on the pre-defined conditions. (i.e., Step level conditional auto-retry)

~~The Jobs arrived to Executor does not contains all the information. Executor will populate all the needed information (to send the requests) from Job.~~

> Policy Engine CAN and WILL if Pillar require to put enough information into the request to pillar for each Job so pillar does not need to fetch again.
>
> Parameter population for other information (e.g., assetId to AssetDetail) will be put into fridge for phase 1. We'll revisit on phase 2 or when we got performance issues.



SDK for enqueue

Move conflict check to dispatcher

Move generic data populate mechanism (Job to Variables)

Pass more data rather than ref in the request to pillars

New monitoring flow

New cancel flow

7.7.1. Phase 1 work

1. API for execute
2. Migrate to spring-reactive framework
3. Clean up the unnecessary cache
4. Optimize variables for pillar request. Pass more data directly (less ref).
5. Optimize the workflow and activity definitions
6. Get rid of workflow resource
7. New monitoring flow
8. New cancel flow (send explicit cancel request to pillar, after received the execution response)
9. SDK for consumer
10. Observability
11. Resilience hardening
12. Periodical pulling pillars for progress update thru spec driven API endpoints + new heart-beat

7.7.2. Resilience targets

Max down time < 1m (time from restart to READY)

7.7.3. Performance Target

	Throughput (jobs per sec)
Job Issuing	500

7.8. Activity

Activity Service is the owner of "activity" resource and "steps" sub-resource. It provides REST APIs for the other components to record the job/task/step information and emit notifications to the subscribers. Customer can use the REST APIs to track the job status and view the details, as well as cancel/retry a single job or in batch.

Activity has two major use cases,

1. To the user: feedback of the job status and a proxy to intervene the job execution (e.g., cancel)

2. To the application: a protocol to let the northbound services (mainly executor) to monitor the execution of a request from southbound services (e.g., VMDM)

When a southbound service got a request from the northbound service, if it's an async request, then it should create an activity (job/task) and return the ID as "handle" for the northbound service to monitor the execution status of this particular request.

Activity Service will do automatic roll-up for any fields if possible (e.g., progress, bytes transferred, etc.).

Activity Service will record the cancel/retry and notify the "owner" of the job for further processing asynchronously.

Activity Service is the most busy service.

Fully API driven. No more CRS.

Dedicate APIs for progress and step

Optimized data persistence for progress and steps

New monitoring flow. Take more responsibilities in automatic progress/status rollup

New retry/cancel flow. No lock during "PENDING_CANCEL". No lock after complete

7.8.1. Phase 1 work

1. Data ownership of "Activity" and REST APIs
2. Migrate to spring-reactive framework
3. Automatic progress/status rollup
4. New cancel/retry flow
5. Redis + merger + flush for transient progress info
6. Dedicate index + append only + aggregation for steps
7. SDK for consumer
8. Observability
9. Migrate all consumers from CRS to SDK. Disable/deprecate Activity CBS/CRS
10. Resilience hardening

7.8.2. Performance Target

	Throughput (rps)	Latency (ms of 99 percentile)
Get by ID	1k	10
Progress Update	1k	10
Step Update	500	200
Others	200	200